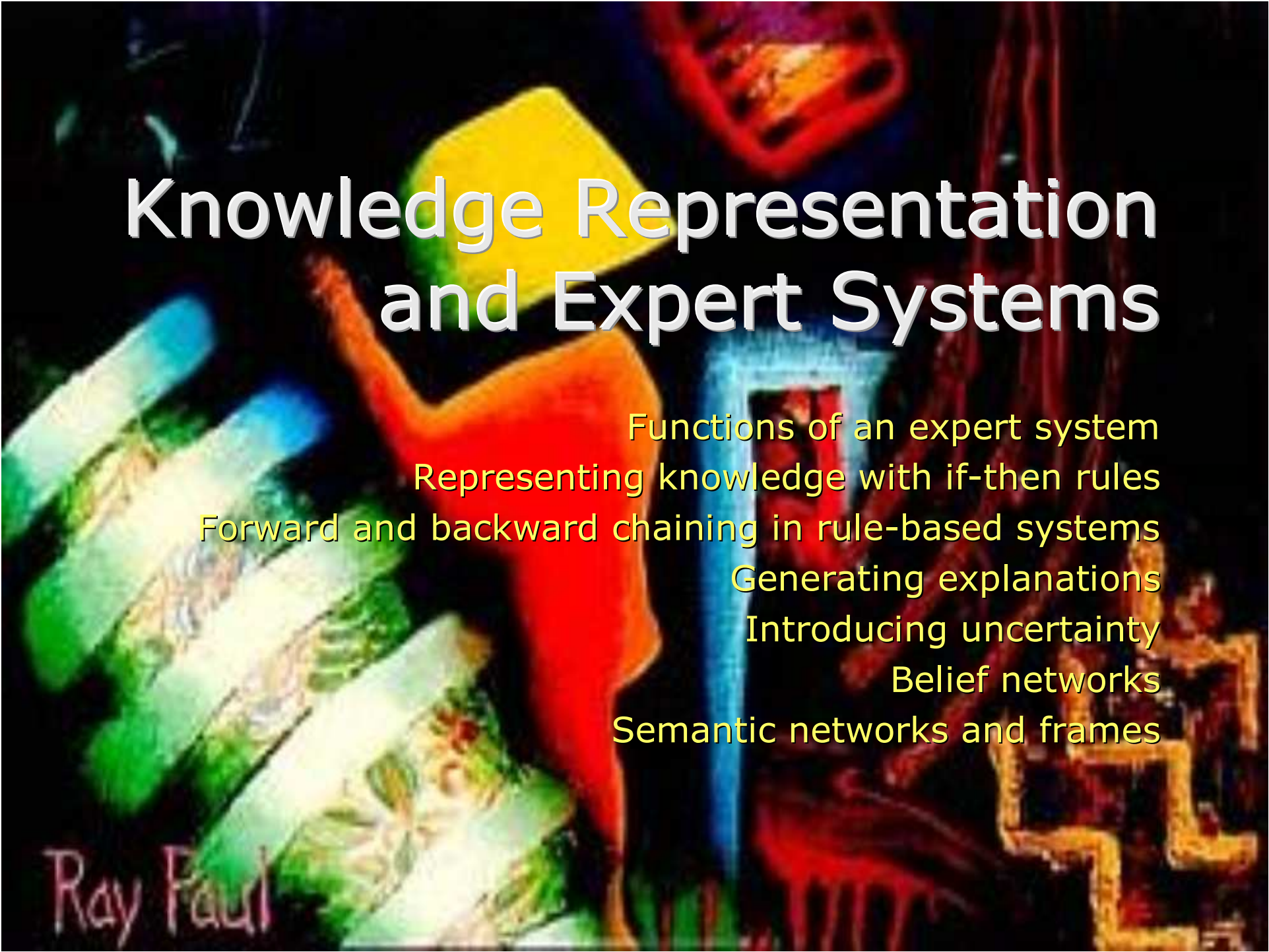


An abstract painting by Ray Paul, featuring a dark background with vibrant, overlapping shapes in yellow, orange, red, blue, and green. The composition is dynamic and expressive, with visible brushstrokes and a sense of movement. The title 'Knowledge Representation and Expert Systems' is overlaid in white text.

Knowledge Representation and Expert Systems

Functions of an expert system

Representing knowledge with if-then rules

Forward and backward chaining in rule-based systems

Generating explanations

Introducing uncertainty

Belief networks

Semantic networks and frames

Ray Paul

Expert Systems

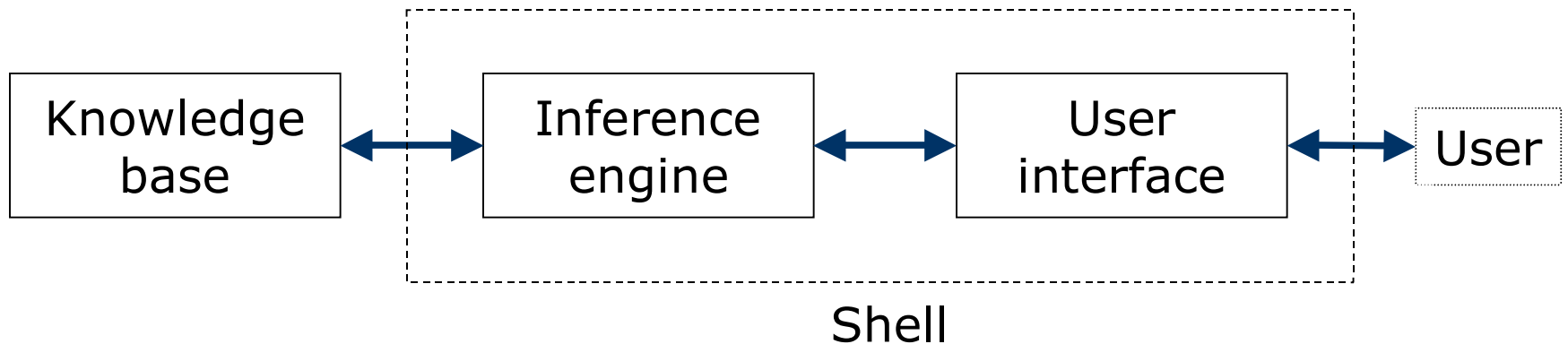
- Programs that behave like an *expert* in some (narrow) domain:
 - medical diagnosis
 - locating equipment failures
 - interpreting measurement data
- Capable of solving problems that require expert knowledge
- Sometimes called *knowledge-based systems*
- Capable of *explaining* its behavior and decisions

Expert System Structure

- Functions

- problem-solving (processing knowledge, dealing with uncertainty)
- user-interaction (dialog, explanation)

- Modules



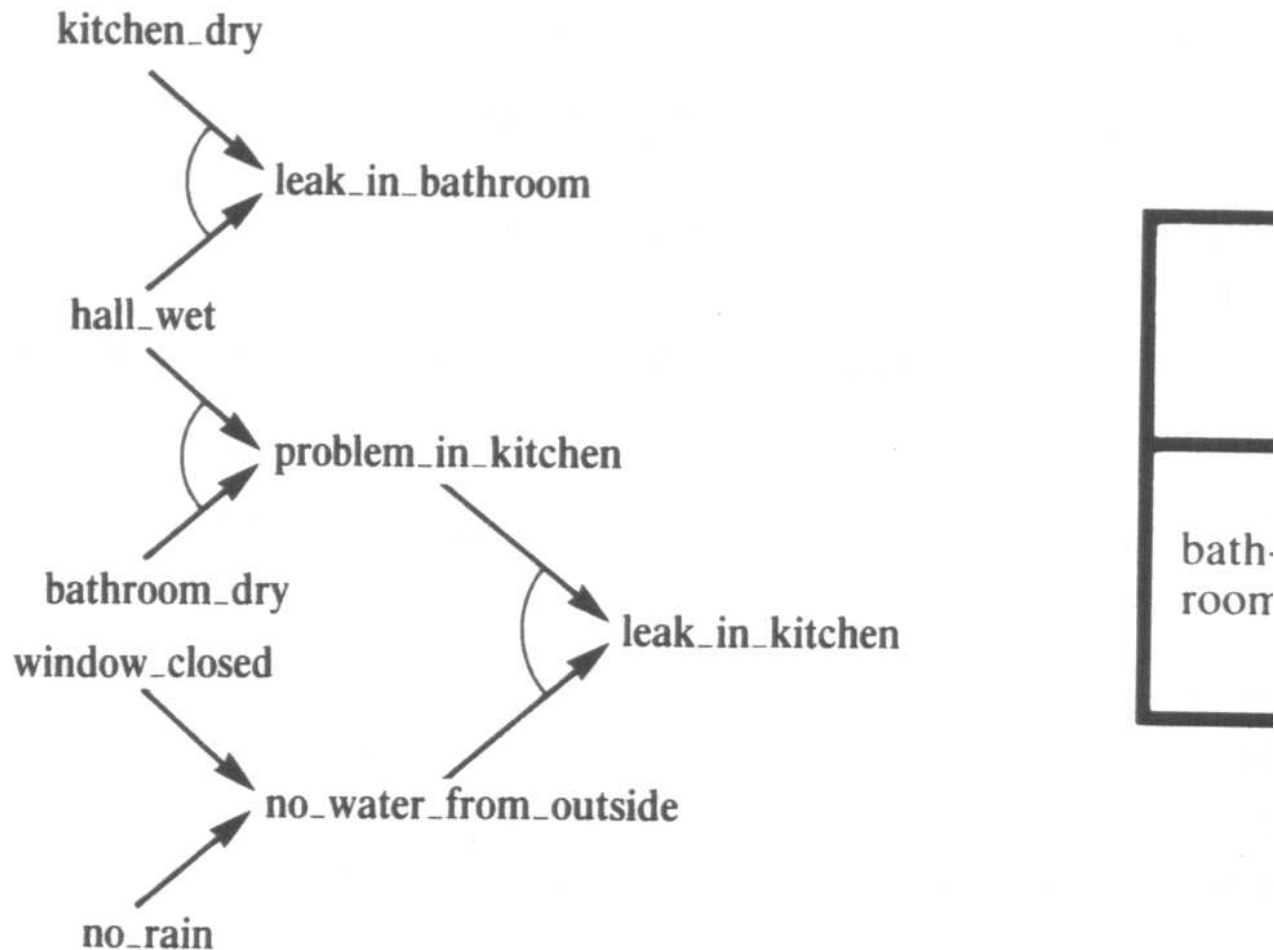
Representation with if-then rules

- Any formalism that can express knowledge about a certain domain can be considered for an ES.
- Popular: if-then rules (also called *production* rules).
- Example
 - *if* precondition *P* *then* conclusion *C*
 - *if* situation *S* *then* action *A*
 - *if* conditions *C1* and *C2* hold *then* condition *C* does not hold

Benefits of if-the rules

- “natural” way to express knowledge
- *Modularity*: each rule defines a small, independent piece of knowledge
- *Incrementability*: new rules can be added
- *Modifiability*: old rules can be removed
- *Transparency*:
 - “**How** did you reach this conclusion?”
 - “**Why** are you interested in this information?”

Example: diagnose water leaks

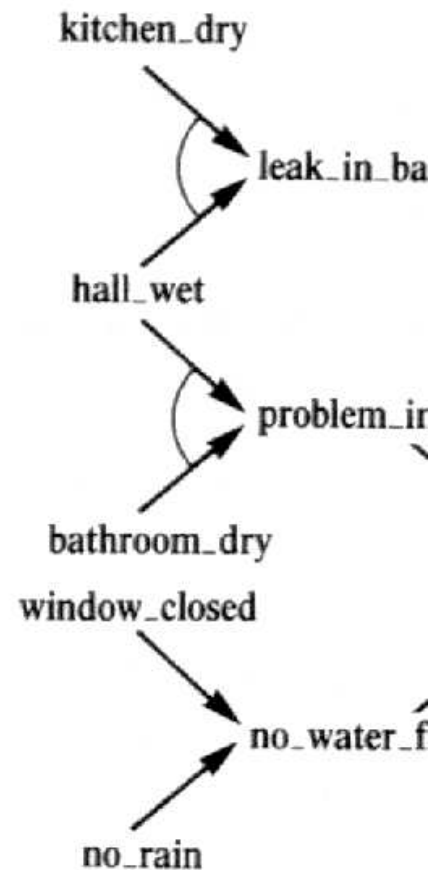


Example: diagnose water leaks

```
if
    hall_wet and kitchen_dry
then
    leak_in_bathroom.
```

```
if
    hall_wet and bathroom_dry
then
    problem_in_kitchen.
```

```
if
    window_closed or no_rain
then
    no_water_from_outside.
```



Reasoning procedure

- Two basic ways of reasoning with if-then rules:
 - backward chaining (goal driven), or
 - forward chaining (data driven).
- Backward: prove hypothesis; start with a hypothesis and work backwards, according to the rules.
- Forward: produce more facts; start with confirmed findings, and produce more conclusions.

Generating explanation

- How?

- ES shows the *proof tree* of how the final conclusion follows from rules and facts in the knowledge base.

- Why?

- ES shows the current *rule* that it tries to prove.

Introducing uncertainty

- Assumption that facts and rules are *categorical* (i.e. either true or false) does not always hold.
- Simple uncertainty scheme:
Proposition: CertaintyFactor
if Condition then Conclusion: Certainty
 $c(P1 \text{ and } P2) = \min(c(P1), c(P2))$
 $c(P1 \text{ or } P2) = \max(c(P1), c(P2))$
if P then Q:C $c(Q) = c(P)*C$

Difficulties in handling uncertainty

- Different methods and theories are actively debated
- Ad-hoc schemes:
 - criticized of being too simple
 - counterintuitive results; e.g. Q if P or R
Consider $c(P)=.5$, $c(R)=0$ and $c(P)=.5$, $c(R)=.5$, no different result (?!).
- Probability theory:
 - mathematically sound, but often impractical
 - requires information about probabilities, which is not always available.
 - Assumption of independency may not hold.

Belief networks

- Also called *Bayesian networks*.
- Operate on conditional probabilities

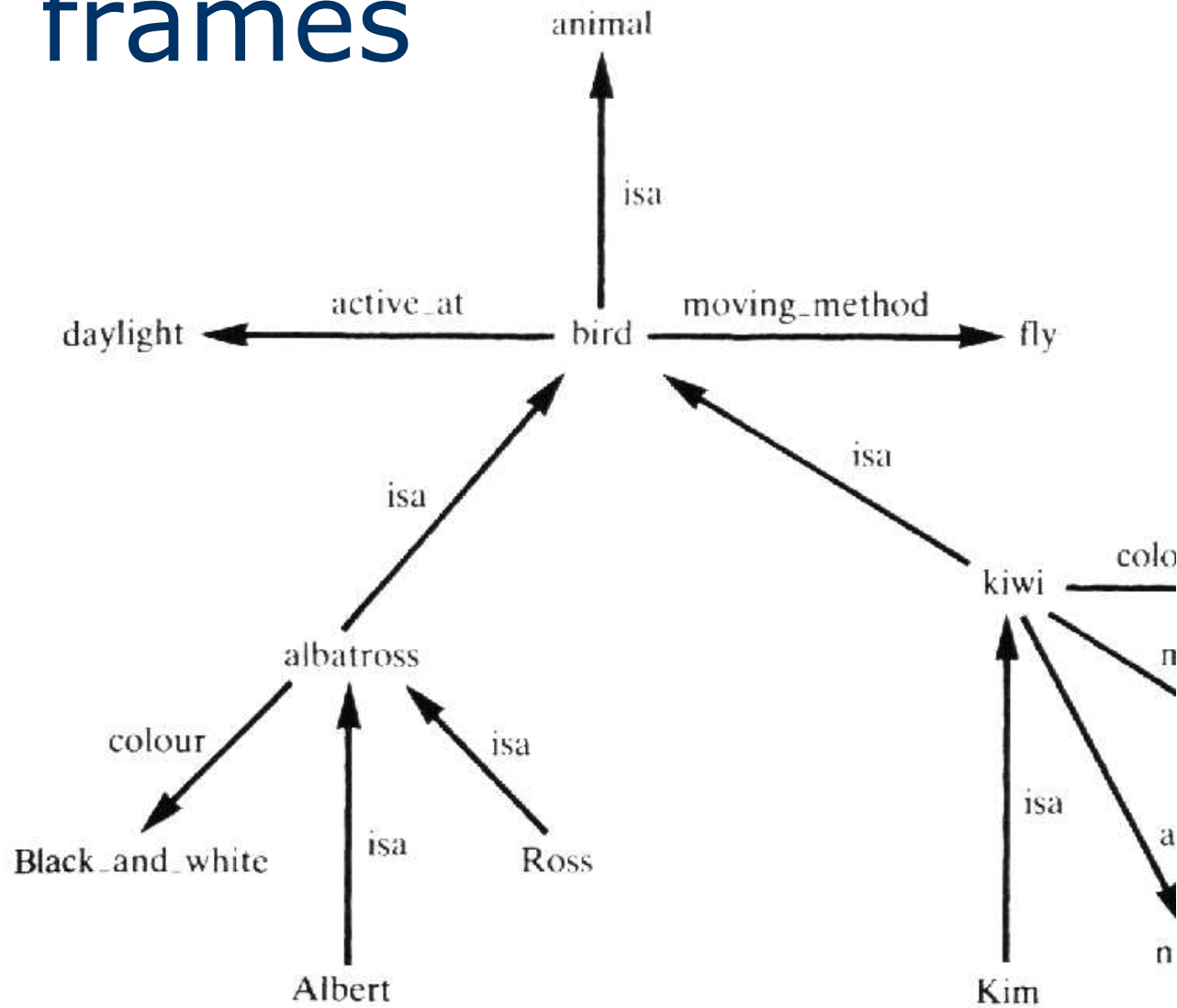
$$p(H|E)$$

H =Hypothesis, E =Evidence

and Bayes' theorem

$$p(X|Y) = p(X) * p(Y|X) / p(Y)$$

Semantic networks and frames



Semantic Networks

- Nodes = entities, links = relations
 - "A bird is a kind of animal."
 - "Flying is the normal moving method of birds."
 - "An albatross is a bird."
 - "Albert is an albatross, and so is Ross."
- Simple Representation in Prolog
 - `isa(bird, animal).`
 - `isa(ross, albatross).`
 - `moving_method(bird, fly).`
 - `moving_method(kiwi, walk).`

Frames

- Facts are clustered around an objects.
- Objects can be concrete physical objects or more abstract concepts such as a class of objects, or a situation.
- *Frame* is a data structure with components called *slots*.

Frames: Examples

- Frame: bird
 - a_kind_of: animal
 - moving_method: fly
 - active_at: day_light
- Frame: albatross
 - a_kind_of: bird
 - color: black_and_white
 - size: 115
- Frame: kiwi
 - a_kind_of: bird
 - moving_method_ walk
 - active_at: night
 - color: brown
 - size: 40